

PWM

TinkerCAD Activity: "PWM"

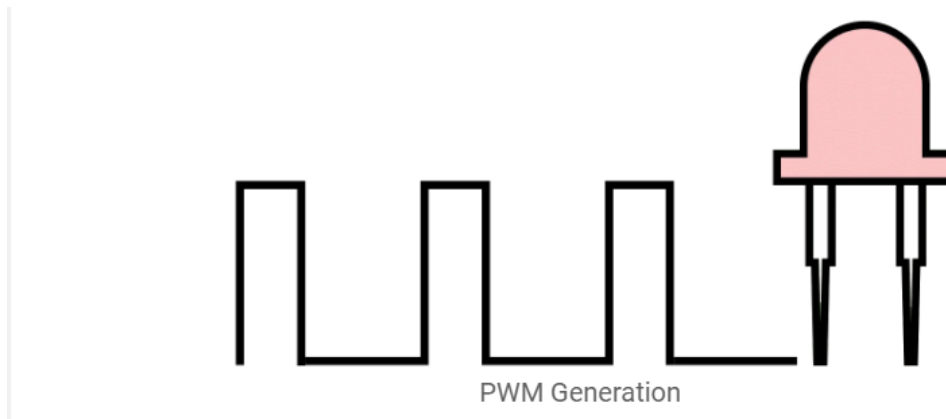
1. Tinker: "1. Fading an LED using PWM"

Introduction

One of the most common applications of PWM in embedded systems and hobby projects is controlling the brightness of an LED. By varying the duty cycle of the PWM signal, we can effectively control the average voltage applied to the LED, and consequently, its brightness.

Objective

Fade an LED in and out, creating a "breathing" effect. It's known that in order to change the intensity of the LED – it's required to change the duty cycle of a PWM signal applied to an LED. This way, you are essentially changing the fraction of time the LED is ON versus the time it is OFF during each PWM cycle – meaning changing the "average intensity" of the LED. The LED turns on and off very rapidly, much faster than the human eye can perceive, so we don't see the blinking but instead perceive a change in brightness.



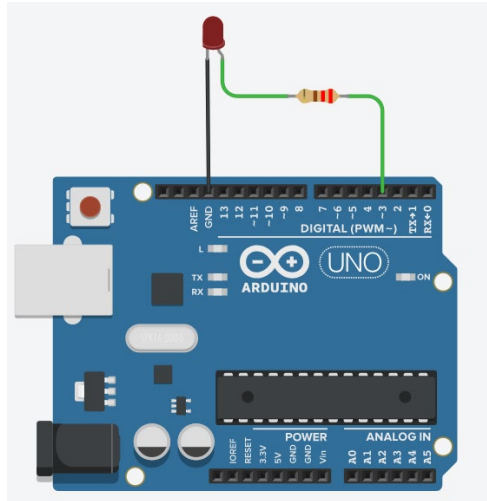
(good animation example: <https://www.electronicwings.com/arduino/pwm-in-arduino>)

Components

- ATmega328P (from Arduino Uno board)
- 1 LED
- 1 220-ohm resistor

Circuit Setup:

- Connect the anode (longer leg) of the LED to one end of the resistor.
- Connect the other end of the resistor to digital pin 3 (OC2B) of the Arduino/ATmega328P.
- Connect the cathode (shorter leg) of the LED to a Ground (GND) pin on the Arduino.



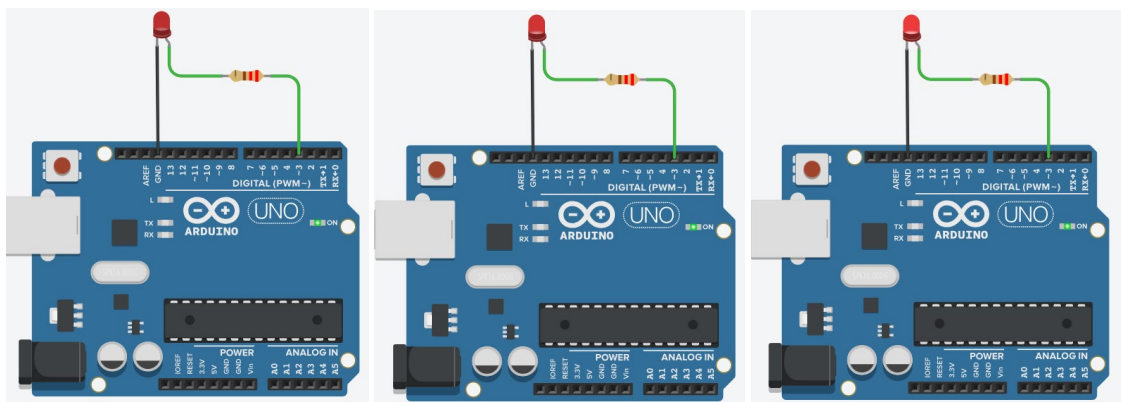
Guidelines:

- Go to “Activities” and “Copy and Tinker” the associated task and update your “submission file name”.
- Setup PWM:** Configure relevant registers and Timer2 to generate the PWM signal.
- Main Loop:** Write a program that should gradually make the LED get brighter up to maximum brightness and then get dimmer until it's OFF (and vice versa).

Accomplish this task by varying gradually the duty cycle of the PWM signal.

- You're allowed to use “delay(10);” during your program.
- You're NOT allowed to use the “analogWrite” function!**

Result Example:



2. Tinker: “2. Controlling Fading LED using Potentiometer”

Introduction

In this exercise, we'll use a potentiometer to manually control the brightness of an LED by adjusting the duty cycle of a PWM signal based on the potentiometer's position.

Components

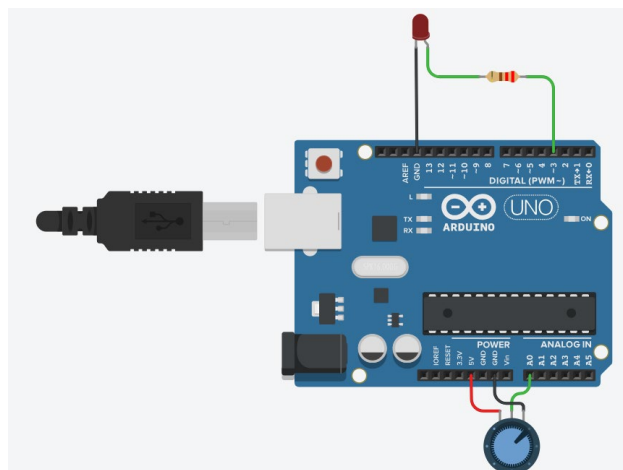
- ATmega328P (from Arduino Uno board)
- 1 LED
- 1 220-ohm resistor
- 1 10k-ohm potentiometer

Circuit Setup

1. **LED Connection:**
 - a. Connect the anode (longer leg) of the LED to one end of the resistor.
 - b. Connect the other end of the resistor to digital pin 3 (OC2B) of the Arduino/ATmega328P.
 - c. Connect the cathode (shorter leg) of the LED to a Ground (GND) pin on the Arduino.
2. **Potentiometer Connection:**
 - a. Connect one of the outer pins of the potentiometer to +5V on the Arduino.
 - b. Connect the other outer pin to Ground (GND).
 - c. Connect the middle pin (wiper) of the potentiometer to analog pin A0 on the Arduino.

Explanation

The potentiometer acts as a voltage divider. As you turn its knob, the voltage at the wiper (middle pin) changes. This voltage is read by the Arduino's analog pin A0. The Arduino then maps this analog reading (which will be between 0 and 1023) to a value suitable for PWM control (between 0 and 255). This value is then used to set the duty cycle of the PWM signal, controlling the brightness of the LED.



Guidelines:

- a. Go to “Activities” and “Copy and Tinker” the associated task and update your “submission file name”.
- b. **Setup PWM:** Configure relevant registers and Timer2 to generate the PWM signal.
- c. **Main Loop - Reading Potentiometer and Adjusting LED Brightness:**
 - a. Read the value from the potentiometer using “analogRead()” (for simplicity).
 - b. Map this value to a range suitable for PWM.
 - c. Adjust the PWM duty cycle based on the mapped value.

3. Tinker: “3. Servo Motor – Using PWM”

Introduction

A servo motor is a rotary actuator that allows for precise control of angular position. The term "Micro Servo" specifically refers to smaller, lightweight servos often used in robotics, RC airplanes, and other applications where compact size and weight are priorities.

Micro Servos are used in “Robotics”, “Toys” (dancing figures, RC cars, airplanes, etc), or even in “Cameras” for adjusting positions, focus, and so on.

How Does a Micro Servo Work?

A micro servo consists of a small motor, a control circuit, and a potentiometer (variable resistor). The potentiometer is connected to the servo's shaft, providing feedback to the control circuit about the shaft's position. This feedback mechanism allows the servo to maintain precise positioning.

Micro servos are typically controlled using PWM. Instead of varying the voltage to control the servo (as you might with a simple motor), you send a series of pulses. The duration of these pulses, the time of the HIGH value, determines the angle at which the servo sets itself.

So if you want to set a servo to a particular angle, you send a signal. For example:

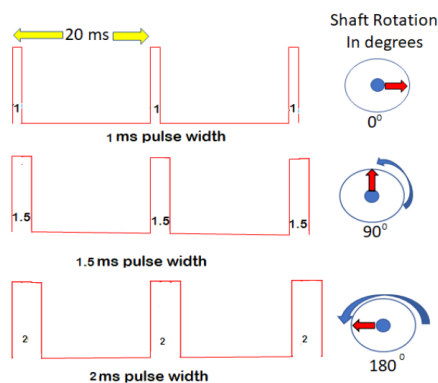
1. 0-degree position – a signal of 1ms may be used.
2. 90-degree position (Neutral) – 1.5ms signal.
3. 180-degree position – 2ms signal.

But note that the signals required for a servo may differ between brands and models (check out datasheet for better specification).

How often does a signal need to be sent? Usually, common servos expect a pulse roughly every 20ms. This doesn't need to be exact but should be within a range that the servo expects (often between 50Hz to 60Hz).

Project Example

The provided program demonstrates how to control a servo motor. As we mentioned, Servos are controlled using a 20ms (50Hz) pulse cycle, and within this cycle, the duration of the HIGH pulse, usually between 1ms and 2ms, determines the servo's angle.



No part of the above may be copied and/or distributed and/or presented publicly and/or given to a third party without the prior written permission of Vladi Sorkin

To implement this in an easier manner, we can consider using the Servo.h library on an Arduino. This abstracts the complexities of generating precise PWM signals for servo control.

Using this library, we'll be able to directly specify the "servo's position" and let the corresponding signal be generated. So in this case, we'll be able to start working in practice more easily without getting into the actual implementation and technicalities.

Let's start.

```
#include <Servo.h> // Include the Servo library
```

This line includes the Servo.h library, which provides functions and methods to easily control servo motors.

```
Servo myServo; // Create a Servo object
```

Then we create a "Servo" object to represent our servo motor and allow us to control it. This object will probably already have all the functionality we need to work with Servo.

```
void setup() {  
  myServo.attach(10); // Attach the servo to Digital Pin 10  
}
```

The setup() function runs once when the Arduino starts. We use the attach() method to specify that our servo is connected to digital pin 10. Note, we specify this information for the "myServo" object we defined previously.

```
void loop() {  
  myServo.write(30); // Set the servo to 30°  
  delay(500);  
  
  myServo.write(60); // Set the servo to 60°  
  delay(500);  
  
  myServo.write(90); // Set the servo to 90° (neutral position for most servos)  
  delay(500);  
}
```

In this example, the servo will move to 30°, wait for half a second, move to 60°, wait for another half a second, and then move to 90°. After reaching 90°, the loop will restart, and the servo will move back to 30°.

Run this project on TinkerCAD and observe the results.

Please NOTE, the code is ready. Nothing needs to be added. It's just a tutorial.

Just complete the usual step of:

Go to “Activities” and “Copy and Tinker” the associated task and update your “submission file name”.

4. Tinker: "4. Smooth Transition for Servo Motor"

Project Example

The provided program demonstrates how to control a servo motor. As we mentioned, Servos are controlled using a 20ms (50Hz) pulse cycle, and within this cycle, the duration of the HIGH pulse, usually between 1ms and 2ms, determines the servo's angle.

Introduction

In this exercise, you will complete the code to create a smooth transition for a servo motor using PWM. The goal is to move the servo smoothly from 0° to 180° and back to 0°.

Task

Complete the code to make the servo motor **move smoothly** between 0° and 180°.

Steps:

1. Use a method to gradually increase the angle from 0° to 180°.
2. Use a method to gradually decrease the angle from 180° to 0°.
3. Add a short delay between each step to ensure smooth movement.

Note:

- The servo should move smoothly, so ensure there are small steps and delays between each angle change.
- Go to "Activities" and "Copy and Tinker" the associated task and update your "submission file name".
- Run this project on TinkerCAD and observe the results.